

Bases de Datos

Práctica 2 - Sesión 1.

- La práctica se debe entregar en **DeliverIt** donde se realizarán una serie de pruebas unitarias automáticas.
- Para que los archivos de código fuente compilen se debe respetar estrictamente los nombres de las clases y paquetes.
- Adicionalmente, el alumno puede y debe comprobar el resultado del código fuente utilizando su propio entorno local.

1. Descripción general

En esta primera práctica se **piden 2 clases**:

BBDDManager. Sirve para gestionar las conexiones a la base de datos y para ejecutar tareas eligiendo modo AC activado o desactivado.

CreateTable. Sirve para ejecutar un comando de SQL que crea la tabla asociada a la relación **imparte**.

Se proporcionan las siguientes clases e interfaces:

1. Una interfaz, **DataBaseTask**, que servirá para definir distintas tareas a realizar en la base de datos. La clase **CreateTable** implementa la interfaz.
2. Una clase derivada de **Exception**, **BBDDException**, para añadir información adicional a otras **Exception** que se puedan producir.
3. Una clase **StringWriter** para escribir los mensajes.

Advertencia Debes usar los archivos de las clases que se proporcionar para empezar a trabajar sobre ellas.

2. Modelo de Datos

Todas las clases, incluidas las clases que se proporcionan, están en un **paquete** que se llama **curso**. El alumno debe escribir todas las clases en ese paquete.

2.1. Clases e interfaces

BBDDException.

```
public class BBDDException extends Exception {

    private String when;

    /**
     * Permite construir una BBDDException a partir de
     * cualquier otra Exception, pero indicando un dato
     * adicional, when, a la Exception que se habia producido.
     *
     * @param e, la Exception original
     * @param when, el momento en que se ha producido
     */
    public BBDDException (Exception e, String when) {
        super(e);
        this.when = when;
    }

    /**
```

```

    * Devuelve when, el dato que se indica adicionalmente
    * a la Exception original.
    * @return when
    */
    public String when() {
        return when;
    }
}

```

```

}

```

DataBaseTask.

```

public interface DataBaseTask {

    /**
     * Realiza una tarea en una base de datos
     *
     * Ejecuta una tarea en la base de datos dada por la conexion.
     * Si la tarea no se puede realizar correctamente
     * se lanza una BBDDException que se contruye y lanza con
     * cualquier otra Exception que se pueda producir.
     *
     * Se deben cerrar todos los Statement o PreparedStatement
     * que se hayan utilizado.
     *
     * PRE: conn abierta correctamente.
     * POST: conn permanece abierta.
     * @param conn
     * @param data
     * @throws SQLException si se produce un error sobre la bbdd.
     * @throws BBDDException si se produce otros errores
     */
    void run(Connection conn, String data) throws BBDDException, SQLException;
}

```

BBDDManager.

```

/**
 * Clase para gestionar las conexiones y tareas de una BBDD.
 *
 * Cuando se construye se fijan los datos de las conexiones: user,
 * password y URL.
 *
 * - La base de datos se llama cursos_db.
 * - La conexion se establece con el localhost
 * - En el puerto por defecto (3306)
 *
 * Cuando se le pide que ejecute tareas (run) crea una conexion
 * y se la pasa a todas ellas, gestionando todas las excepciones
 * que se puedan producir.
 */
public class BBDDManager {

    // Declara los atributos necesarios.

    /**
     * Construye un gestor de conexion.
     *
     * @param user, el usuario a emplear en la conexion
     * @param password, su password
     */
    public BBDDManager(String user, String password) { ... }

    /**

```

```

    * Devuelve la URL que se emplea en la conexi3n.
    *
    * @return La url completa para conectar
    */
public String url() { ... }

/**
 * Ejecuta una secuencia de tareas dada por tasks utilizando
 * para cada una los datos en dataArray y con el modo autocommit indicado.
 *
 * 1) Abrir una conexi3n (con la url, el usuario y el passwd),
 * 2) Se ejecutan todas las tareas cada una con su correspondiente
 *    String en dataArray
 * 3) Es obligatorio cerrar la conexi3n.
 *
 * Se deben capturar las excepciones de manera que:
 *
 * + Los distintos errores que se puedan producir se concatenan en la
 *   String que se retorna.
 *
 * - Si se produce un error SQLException al hacer la conexi3n o al fijar
 *   el autocommit se concatena al String "Connection:" + e.getMessage() + ";"
 * - Si se produce cualquier otra Exception se concatena
 *   al String "Otro:" + e.getMessage() + ";"
 * - Si se produce una BBDDException al realizar una tarea
 *   se concatena al String: "Task:" + e.when() + ";" + e.getMessage() + ";";
 * - Si se produce una SQLException al realizar una tarea
 *   se concatena al String: "SQL:" + e.getMessage() + ";";
 *
 * Adicionalmente, si la Exception que se captura es SQLException se hara un rollback
 * y si es BBDDException se hara commit. Se asume que ni commit ni rollback producen
 * excepciones cuando autocommit es false.
 *
 * Las String que se retornan NO TIENEN ESPACIOS ADICIONALES.
 *
 * Al terminar se debe concatenar "fin".
 *
 * Este metodo no debe lanzar _ninguna_ Exception
 *
 * @param tasks, un array con la tareas a realizar
 * @param dataArray, un array con los datos para las tareas
 * @param autoCommit, el modo de ejecuci3n de las tareas sobre la bbdd.
 * @return El resultado de concatenar los distintos mensajes que se puedan producir.
 */
public StringWriter run(DataBaseTask[] tasks, String[] dataArray, boolean autoCommit) { ... }
}

```

Advertencias

1. Los nombres en SQL son *case-sensitive* porque el servidor de Deliverit es un Linux. Es especialmente importante que el nombre de la tabla sea **imparte** en minúsculas.
2. El código en Java debe escribirse en ISO-8859-1. Para evitar problemas lo que sencillo es evitar todo tipo de caracteres especiales (tildes, eñes) en todo el contenido de los fuentes (incluyendo comentarios).

2.2. Programa para comprobar localmente

Para poder comprobar localmente es necesario crear una base de datos `cursos_db` que tenga un usuario `alumno` con contraseña `'bbdd-upm'`. Puedes usar el programa en `Main.java` para pruebas preliminares. **IMPORTANTE:** Las pruebas en DeliverIt no usan este programa sino otra versi3n adecuada para hacer las pruebas unitarias.